

## UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

### Topic 03 – Matrix structure, factorizations

- 1 Special Matrices
  - Covariance and correlation matrices
  - Triangular matrices
  - Banded matrices
  - Toeplitz matrices
  - Symmetric positive-definite

- Diagonal dominance
- Orthogonal matrices
- 2 Matrix factorizations
  - LU factorization
  - Cholesky factorization
  - QR factorization
  - Singular value decomposition

# Covariance and correlation matrices

Consider a vector-valued random variable  $\mathbf{X} = (X_1, \dots, X_n)^T$ .

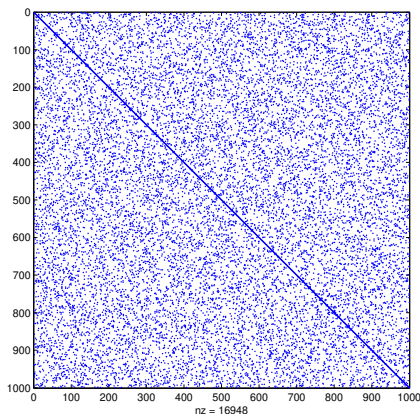
- **Mean:**  $\mu_i = \mathbb{E}[X_i]$  for  $i = 1, \dots, n$
- **Variances:**  $\sigma_i^2 = \mathbb{E}[(X_i - \mu_i)^2]$  for  $i = 1, \dots, n$
- **Covariance** of  $X_i$  and  $X_j$ :  $C_{ij} = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)]$ .

Covariance measures whether two variables tend to increase or decrease together.

- **Covariance matrix:**  $C = [C_{ij}] \in \mathbb{R}^{n \times n}$ 
  - **Interpretation:**  $C_{ii} = \sigma_i^2$  is the variance of  $X_i$ ;  $C_{ij} > 0$  if  $X_i$  and  $X_j$  vary in the same direction;  $C_{ij} < 0$  if they vary in opposite directions.
  - **Properties:**  $C$  is symmetric and positive semidefinite.
- **Correlation matrix:**  $K = [K_{ij}] \in \mathbb{R}^{n \times n}$ ,  $K_{ij} = \frac{C_{ij}}{\sigma_i \sigma_j}$ ,  $i, j = 1, \dots, n$ .
- **Properties:**
  - $K = \Sigma^{-1} C \Sigma^{-1}$ ,  $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n) \in \mathbb{R}^{n \times n}$  is a **diagonal** matrix
  - $K_{ii} = 1$  for  $i = 1, \dots, n$ , and  $|K_{ij}| \leq 1$  for all  $i, j$ . Thus,  $K$  is unit-free and easier to interpret.
- **Negative correlation** means the two variables tend to vary in opposite directions.

# Correlation matrix

Correlation matrix: `corr1000.mat`



- MATLAB `spy` plots the `nonzero` elements;
- Python `matplotlib.pyplot.spy`

# Triangular matrices

- Many numerical methods used in finance rely heavily on linear algebra.
- We often deal with matrices with some special structure.
- Exploiting such structure may lead to dramatic improvements in computational efficiency (both storage and time), and can sometimes also improve accuracy.
- Example: `stex.m`, `stex.py`

## Definition (Triangular matrices)

An  $m \times n$  matrix  $A = [a_{ij}]$  is

- **upper triangular** if  $a_{ij} = 0$  for all  $i > j$ ;
- **lower triangular** if  $a_{ij} = 0$  for all  $i < j$ .
- A **square** lower (upper) triangular matrix  $A$  is **unit** lower (upper) triangular if  $a_{ii} = 1$  for all  $i$ .

# Triangular matrices (continued)

## Example (Triangular matrices)

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ -3 & 0 & 1 & 0 \\ 2 & 7 & -1 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 2 & 0 & -5 & 6 & 10 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 4 & 2 & 8 \\ 0 & 0 & 0 & 7 & 3 \end{bmatrix}$$

- $L$  is unit lower triangular;
- $U$  is upper triangular (but not unit upper triangular).

If  $A = [a_{ij}] \in \mathbb{R}^{n \times n}$  is triangular, then

- $\det(A) = a_{11}a_{22} \cdots a_{nn}$
- $A$  is invertible (nonsingular) iff  $a_{ii} \neq 0$  for  $1 \leq i \leq n$ .

# Triangular matrices (continued)

We solve a lower triangular linear system by **forward substitution** and an upper triangular linear system by **back substitution**.

## Example (Forward substitution)

Solve

$$\begin{array}{rclclcl} x_1 & & & & & = & 3, \\ 3x_1 & + & x_2 & & & = & 9, \\ -3x_1 & & & + & x_3 & = & -11, \\ 2x_1 & + & 7x_2 & - & x_3 & + & x_4 = 12, \end{array}$$

Solution:  $x_1 = 3, \quad x_2 = 9 - 3x_1 = 0, \quad x_3 = -11 + 3x_1 = -2,$   
 $x_4 = 12 - 2x_1 - 7x_2 + x_3 = 12 - 6 - 0 - 2 = 4.$

In general, for a unit lower triangular matrix  $L = [L_{ij}] \in \mathbb{R}^{n \times n}$ , **forward substitution** to solve  $L\mathbf{x} = \mathbf{b}$  takes about  $n^2$  flops:

$$x_1 = b_1, \quad x_i = b_i - \sum_{j=1}^{i-1} L_{ij}x_j \quad \text{for } i = 2, \dots, n.$$

# Triangular matrices (continued)

We solve square linear systems of the form  $Ax = b$  in MATLAB as

$$x = A \backslash b,$$

and in Python as

$$x = \text{numpy.linalg.solve}(A, b).$$

Some properties of triangular matrices: Let  $A$  be a square matrix,

- $A$  is upper triangular iff  $A^T$  is lower triangular;
- the product of two unit lower triangular matrices is again unit lower triangular;
- the product of two upper triangular matrices is again upper triangular;
- the inverse of a (unit) lower triangular matrix is (unit) lower triangular;
- the inverse of a nonsingular upper triangular matrix is upper triangular.

# Banded matrices

## Definition (Banded matrices)

A matrix  $A = [a_{ij}]$  has

- **lower bandwidth**  $m_l$  if  $a_{ij} = 0$  for all  $i - j > m_l$ ;
- **upper bandwidth**  $m_u$  if  $a_{ij} = 0$  for all  $j - i > m_u$ ;
- **total bandwidth**  $m_l + m_u + 1$ , the number of potentially nonzero diagonals.

## Example (Banded matrix)

$$\begin{bmatrix} 3 & 0 & -1 & 0 & 0 & 0 & 0 \\ -1 & -2 & 0 & 2 & 0 & 0 & 0 \\ 5 & 8 & 5 & 0 & 9 & 0 & 0 \\ -2 & 0 & 2 & -7 & 0 & -1 & 0 \\ 0 & 1 & 3 & 6 & -4 & 0 & -2 \\ 0 & 0 & 7 & -3 & 1 & 5 & 0 \end{bmatrix}$$

then  $m_l = 3$ ,  $m_u = 2$  and the total bandwidth is 6.

# Special banded matrices

## Definition (Diagonal, tridiagonal matrices)

A square matrix is

- **diagonal** if  $m_l = m_u = 0$ , that is,  $a_{ij} = 0$  for all  $i \neq j$ ;
- **tridiagonal** if  $m_l = m_u = 1$ , that is,  $a_{ij} = 0$  whenever  $|i - j| > 1$ .

**Storage** for an  $n \times n$  banded matrix: The number of nonzero elements is at most:

$$\begin{aligned} n + \sum_{i=1}^{m_l} (n - i) + \sum_{j=1}^{m_u} (n - j) &= n + m_l n - \sum_{i=1}^{m_l} i + m_u n - \sum_{j=1}^{m_u} j \\ &= (1 + m_l + m_u)n - \frac{m_l(m_l + 1)}{2} - \frac{m_u(m_u + 1)}{2}. \end{aligned}$$

If  $m_l \ll n$  and  $m_u \ll n$ , this is roughly  $(1 + m_l + m_u)n$ .

## Example (Storage for tridiagonal matrix)

A  $10000 \times 10000$  real, double precision, tridiagonal matrix requires about  $8 \times 3 \times 10000$  bytes  $\approx 240$  KB of storage, compared with about  $8 \times 10000^2$  bytes  $\approx 800$  MB for a dense matrix of the same size.

# Toeplitz matrices

## Definition (Toeplitz matrix)

A matrix  $A = [a_{ij}]$  is **Toeplitz** if  $a_{ij} = \alpha_{j-i}$  for all  $i$  and  $j$ , so the entries are constant along each diagonal.

**Example:** The MATLAB command

```
A=toeplitz([2,1,0,9,2],[2,3,5,0,1,7])
```

or the Python command

```
A=scipy.linalg.toeplitz([2,1,0,9,2],[2,3,5,0,1,7])
```

produces the matrix

$$A = \begin{bmatrix} 2 & 3 & 5 & 0 & 1 & 7 \\ 1 & 2 & 3 & 5 & 0 & 1 \\ 0 & 1 & 2 & 3 & 5 & 0 \\ 9 & 0 & 1 & 2 & 3 & 5 \\ 2 & 9 & 0 & 1 & 2 & 3 \end{bmatrix}$$

# Symmetric and skew symmetric matrices

## Definition

A **real** matrix  $A = [a_{ij}] \in \mathbb{R}^{n \times n}$  is

- **symmetric** if  $A^T = A$ , equivalently  $a_{ji} = a_{ij}$  for all  $i$  and  $j$
- **skew-symmetric** if  $A^T = -A$ , equivalently  $a_{ji} = -a_{ij}$  for all  $i$  and  $j$ .  
Then,  $a_{ii} = 0$  for all  $i$ .

Let  $A \in \mathbb{R}^{n \times n}$  be any **real** matrix.

- $A$  can be decomposed as:  $A = A_{\text{sym}} + A_{\text{skew}}$ , where
  - $A_{\text{sym}} = \frac{A + A^T}{2}$  is the **symmetric part**,
  - $A_{\text{skew}} = \frac{A - A^T}{2}$  is the **skew-symmetric part**.
- If  $A$  is skew symmetric, then the **quadratic form**  $\mathbf{x}^T A \mathbf{x} = 0$  because
 
$$2\mathbf{x}^T A \mathbf{x} = \mathbf{x}^T A \mathbf{x} + (\mathbf{x}^T A \mathbf{x})^T = \mathbf{x}^T A \mathbf{x} + \mathbf{x}^T A^T \mathbf{x} = \mathbf{x}^T (A + A^T) \mathbf{x} = 0.$$
- If  $A$  is any matrix, then  $\mathbf{x}^T A \mathbf{x} = \mathbf{x}^T A_{\text{sym}} \mathbf{x}$ . Hence,  $\mathbf{x}^T A \mathbf{x}$  depends only on  $A_{\text{sym}}$ .

# Positive and negative definiteness

A matrix  $A \in \mathbb{R}^{n \times n}$  is

- (strictly) positive-definite if  $\mathbf{x}^T A \mathbf{x} > 0$  for all  $\mathbf{x} \in \mathbb{R}^n, \mathbf{x} \neq \mathbf{0}$ ;
- positive semi-definite if  $\mathbf{x}^T A \mathbf{x} \geq 0$  for all  $\mathbf{x} \in \mathbb{R}^n$ ;
- (strictly) negative definite if  $\mathbf{x}^T A \mathbf{x} < 0$  for all  $\mathbf{x} \in \mathbb{R}^n, \mathbf{x} \neq \mathbf{0}$ ;
- negative semi-definite if  $\mathbf{x}^T A \mathbf{x} \leq 0$  for all  $\mathbf{x} \in \mathbb{R}^n$ ;
- indefinite if  $\mathbf{x}^T A \mathbf{x} > 0$  and  $\mathbf{y}^T A \mathbf{y} < 0$  for some vectors  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ .

If  $A$  is a real symmetric matrix, then the eigenvalues  $\lambda_1, \lambda_2, \dots, \lambda_n$  of  $A$  are real ( $\lambda$  is an eigenvalue of  $A$  if  $A\mathbf{v} = \lambda\mathbf{v}$  for some nonzero vector  $\mathbf{v}$ ).

Hence,

- $A$  is positive-definite iff  $\lambda_i > 0$  for  $i = 1, \dots, n$
- $A$  is positive semi-definite iff  $\lambda_i \geq 0$  for  $i = 1, \dots, n$
- $A$  is negative definite iff  $\lambda_i < 0$  for  $i = 1, \dots, n$
- $A$  is negative semi-definite iff  $\lambda_i \leq 0$  for  $i = 1, \dots, n$
- $A$  is indefinite iff it has at least one positive and one negative eigenvalue

# Diagonal dominance

## Definition

A real matrix  $A = [a_{ij}] \in \mathbb{R}^{n \times n}$  is **strictly diagonally dominant** if

$$a_{ii} > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \text{for all } i = 1, \dots, n$$

- If  $A$  is symmetric and strictly diagonal dominant, then  $A$  is positive-definite.
- If  $A$  is symmetric and positive-definite, it need not be strictly diagonal dominant.

**Example.** The symmetric matrix  $A = \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}$  is positive-definite because

$$\mathbf{x}^T A \mathbf{x} =$$

$$\begin{aligned} [x \ y] \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= [x \ y] \begin{bmatrix} x + 2y \\ 2x + 5y \end{bmatrix} = x^2 + 2xy + 2xy + 5y^2 \\ &= (x + 2y)^2 + y^2 > 0 \text{ whenever } \mathbf{x} \neq \mathbf{0}. \end{aligned}$$

However,  $A$  is **not strictly diagonally dominant**.

# Orthogonal matrix, $A = [a_{ij}] \in \mathbb{R}^{n \times n}$

$A$  is **orthogonal** if  $A^T A = I_n$  ( $I_n \in \mathbb{R}^{n \times n}$  is the identity matrix)



$$A = [a_{ij}] = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n], \quad \mathbf{a}_j = \text{the } j\text{th column of } A.$$

$A$  is orthogonal **iff** the vectors  $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n$  are **orthonormal**:

$$\mathbf{a}_j^T \mathbf{a}_k = \mathbf{a}_j \cdot \mathbf{a}_k = \delta_{jk} = \begin{cases} 1, & \text{if } j = k, \\ 0, & \text{if } j \neq k. \end{cases}$$

- If  $A$  is orthogonal, then the column vectors  $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n$  are **orthogonal** ( $\mathbf{a}_i \cdot \mathbf{a}_j = 0$  whenever  $i \neq j$ ); hence they are **linearly independent**, so  $\text{rank}(A) = n$ .
- If  $A$  is orthogonal, then it is nonsingular and

$$A^{-1} = I_n A^{-1} = (A^T A) A^{-1} = A^T (A A^{-1}) = A^T.$$

Therefore,  $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \|A\|_2 \|A^T\|_2 = \|A\|_2^2 = 1$  (ideal).

**Note:** For  $\mathbf{x} \in \mathbb{R}^n$ , the 2-norm  $\|\mathbf{x}\|_2$  is also called the **Euclidean norm** of  $\mathbf{x}$ .

# Orthogonal matrix (continued)

$A$  is orthogonal  $\iff (\mathbf{Ax}) \cdot (\mathbf{Ay}) = \mathbf{x} \cdot \mathbf{y}$  for all  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ .

- To show one implication, assume that  $A$  is orthogonal. Then

$$(\mathbf{Ax}) \cdot (\mathbf{Ay}) = (\mathbf{Ax})^T (\mathbf{Ay}) = \mathbf{x}^T (A^T A) \mathbf{y} = \mathbf{x}^T \mathbf{y} = \mathbf{x} \cdot \mathbf{y}.$$

- To show the converse, let  $\mathbf{e}_i \in \mathbb{R}^n$  be the  $i$ th canonical basis vector, so its  $i$ th component is 1 and all others are 0. Let

$$A^T A = B = [b_{ij}].$$

We claim that  $B = I_n$ , that is,  $b_{ij} = \delta_{ij}$ . Indeed,

$$b_{ij} = \mathbf{e}_i^T B \mathbf{e}_j = \mathbf{e}_i^T A^T A \mathbf{e}_j = (\mathbf{Ae}_i)^T (\mathbf{Ae}_j) = (\mathbf{Ae}_i) \cdot (\mathbf{Ae}_j) = \mathbf{e}_i \cdot \mathbf{e}_j = \delta_{ij}.$$

- In fact, the following are equivalent: for all  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

- $A$  is orthogonal  $\iff (\mathbf{Ax}) \cdot (\mathbf{Ay}) = \mathbf{x} \cdot \mathbf{y} \iff \|\mathbf{Ax}\|_2 = \|\mathbf{x}\|_2$ .

# Permutation matrix

A **permutation matrix**  $P \in \mathbb{R}^{n \times n}$  is obtained by permuting the rows of  $I_n$ . The matrix  $P$  is orthogonal, that is,  $P^T P = I_n = P P^T$

- Applying the permutation to the rows of a general  $A \in \mathbb{R}^{n \times n}$  gives  $PA$ .
- MATLAB / Python:  $p$  a permutation of  $[1:n]$ ,  $A(p, :)$  /  $A[p, :]$  gives the same result as  $PA$ .

**Example.** Swap rows 2 and 4 of the matrix  $A = [a_{ij}] \in \mathbb{R}^{4 \times 4}$  using a permutation matrix

$$I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \text{permutation vector } p = [1 \ 4 \ 3 \ 2]$$

$$PA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{41} & a_{42} & a_{43} & a_{44} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{21} & a_{22} & a_{23} & a_{24} \end{bmatrix}.$$

- MATLAB `A([1 4 3 2], :)`; Python `A[numpy.array([1, 4, 3, 2]),:]`

# LU factorization, $A \in \mathbb{R}^{n \times n}$

## Definition

We say that  $A = LU$  is an **LU-factorization** of  $A$  if  $L \in \mathbb{R}^{n \times n}$  is unit lower triangular and  $U \in \mathbb{R}^{n \times n}$  is upper triangular.

If  $A = LU$ , then  $A\mathbf{x} = L\mathbf{U}\mathbf{x} = L\mathbf{y}$ . Hence, we solve  $A\mathbf{x} = \mathbf{b}$  as:

- 1 solve first  $L\mathbf{y} = \mathbf{b}$  for  $\mathbf{y}$  by forward substitution;
- 2 then solve  $U\mathbf{x} = \mathbf{y}$  for  $\mathbf{x}$  by back substitution.

This algorithm is equivalent to Gaussian elimination.

## Proposition (LU factorization is unique)

*When it exists, the LU-factorization of a (nonsingular)  $A$  is unique.*

**Proof.** If  $L_1 U_1 = L_2 U_2$  then  $L_2^{-1} L_1 = U_2 U_1^{-1}$  is simultaneously unit lower triangular and upper triangular. This means

$$L_2^{-1} L_1 = I_n = U_2 U_1^{-1},$$

implying  $L_1 = L_2$  and  $U_1 = U_2$ .

# LU factorization (continued)

Example ( $A = LU$  is not always possible)

$$\begin{bmatrix} 0 & 1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \ell_{21} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} \\ \ell_{21}u_{11} & \ell_{21}u_{12} + u_{22} \end{bmatrix}$$

This is impossible, since  $u_{11} = 0$  contradicts  $\ell_{21}u_{11} = 1$ .

However, swapping rows we have (trivially)

$$\begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}.$$

In general, for any  $A \in \mathbb{R}^{n \times n}$ , a procedure called **partial pivoting** may produce a permutation matrix  $P$ , a unit lower triangular matrix  $L$ , and an upper triangular matrix  $U$  such that

$$PA = LU.$$

# LU factorization (continued)

If  $A$  is nonsingular, we solve  $A\mathbf{x} = \mathbf{b}$  using a partial pivoting  $LU$  factorization as follows:

$$PA\mathbf{x} = P\mathbf{b} \implies LU\mathbf{x} = P\mathbf{b}.$$

Let  $\mathbf{y} = U\mathbf{x}$ . Then  $L\mathbf{y} = P\mathbf{b}$ .

- ① First solve  $L\mathbf{y} = P\mathbf{b}$  for  $\mathbf{y}$  by forward substitution;
  - ② then solve  $U\mathbf{x} = \mathbf{y}$  for  $\mathbf{x}$  by back substitution.
- With  $L = [\ell_{ij}]$ , partial pivoting ensures that  $|\ell_{ij}| \leq 1$  for all  $i > j$ , which helps control round-off errors.
  - Computing  $PA = LU$  costs about  $2n^3/3$  flops.
  - Solving the two triangular linear systems costs a further  $2n^2$  flops.
  - The overall memory requirement is  $8(n^2 + n)$  bytes, assuming double precision real arithmetic.
  - Using the LU-factorization to solve  $A\mathbf{x} = \mathbf{b}$  is faster than computing  $A^{-1}$  and then forming  $\mathbf{x} = A^{-1}\mathbf{b}$ .

MATLAB `LUexbig.m`; Python `LUexbig.py`

# LU factorization (continued)

## Example (LU MATLAB-Python discussion)

### MATLAB `LUex.m`

```
A = rand(4);
[L,U,P]=lu(A)
L=
    1.000         0         0         0
    0.992     1.000         0         0
    0.892    -0.325     1.000         0
    0.139    -0.455     0.257     1.000
U =
    0.913     0.547     0.971     0.142
         0    -0.445     0.002     0.345
         0         0     0.092     0.943
         0         0         0     0.696
```

### Python `LUex.py`

```
from numpy.random import rand
from scipy.linalg import lu

A = rand(4,4)
PT, L, U = lu(A)
P = PT.T

P =
    0         0         0         1
    0         1         0         0
    1         0         0         0
    0         0         1         0
```

# Cholesky factorization

If  $A \in \mathbb{R}^{n \times n}$  is **symmetric** and **positive-definite**, then we may alternatively use the **Cholesky factorization**,

$$A = R^T R,$$

where  $R$  is upper-triangular. To solve the linear system  $A\mathbf{x} = \mathbf{b}$ , we write  $A\mathbf{x} = R^T R\mathbf{x} = R^T \mathbf{y}$ . So,

- ① solve  $R^T \mathbf{y} = \mathbf{b}$  for  $\mathbf{y}$  by forward substitution (costs  $n^2$  flops);
  - ② then solve  $R\mathbf{x} = \mathbf{y}$  for  $\mathbf{x}$  by back substitution (costs  $n^2$  flops).
- Computing  $R$  costs  $n^3/3$  flops (half the cost of **LU** factorization).
  - Cholesky factorization does not require re-ordering the rows/equations (no permutation matrix  $P$ ).
  - The existence of a Cholesky factorization is an efficient way to check whether  $A$  is **positive-definite**. MATLAB: `[R, p] = chol(A)`;  $p = 0$  indicates that  $A$  is positive-definite. Python: `scipy.linalg.cholesky(A)`.

# Cholesky factorization (continued)

## Example (Cholesky MATLAB-Python discussion)

### MATLAB `CHOLex.m`

```

B = rand(4,3);
A = B'*B; % a Sym. 3 by 3 matrix
[R, p] = chol(A);
err = norm(A - R'*R,2);
A =
    4.2406    -0.1634    -0.0332
   -0.1634     3.4503     2.8959
   -0.0332     2.8959     2.6904
R =
    2.0593    -0.0794    -0.0161
         0     1.8558     1.5598
         0         0     0.5072
p = 0
err = 4.4409e-16

```

### Python `CHOLex.py`

```

from numpy.random import rand
from scipy.linalg import cholesky
from scipy.linalg import norm

B = rand(4,3)
A = B.T @ B
R = cholesky(A)
err = norm(A - R.T @ R,2)

```

# QR factorization, $A \in \mathbb{R}^{m \times n}$ with $m \geq n$

If  $\text{rank}(A) = n$ , then  $A$  has a **QR factorization** consisting of an orthogonal matrix  $Q \in \mathbb{R}^{m \times m}$  and a nonsingular upper triangular matrix  $R = [r_{ij}] \in \mathbb{R}^{n \times n}$ , such that

$$A = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = [\textcolor{red}{Y} \quad \textcolor{red}{Z}] \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = YR + Z\mathbf{0} = YR.$$

The zero matrix  $\mathbf{0} \in \mathbb{R}^{(m-n) \times n}$ ,  $Y \in \mathbb{R}^{m \times n}$  and  $Z \in \mathbb{R}^{m \times (m-n)}$ .

We call  $A = YR$  the “economy-size” QR factorization.

If  $A$  is **rank-deficient** ( $\text{rank}(A) < n$ ), then **column pivoting** is required:

$$AP = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = [Y \quad Z] \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = YR,$$

where  $P$  is a column permutation matrix chosen to ensure

$$|r_{11}| \geq |r_{22}| \geq \cdots \geq |r_{nn}|.$$

**Note:** In exact arithmetic, the rank of  $A$  is the number of nonzero diagonal entries of  $R$ ; in numerical computations, very small diagonal entries indicate numerical rank deficiency.

The “economy-size” QR factorization costs about  $2mn^2 - \frac{2}{3}n^3$  flops.

# QR factorization (continued)

## Example (QR MATLAB-Python discussion)

### MATLAB `QRex.m`

```
% A is a 4-by-3 matrix
% C for column
% C1 & C2 are created rand
% C3=2*C1-0.3*C2
A=rand(4,2);
A(:,3)=2*A(:,1)-0.3*A(:,2);
r = rank(A)
[Y, R, p]=qr(A, 0);
err = norm(A(:,p) - Y*R,2);
orthchk=norm(Y'*Y-eye(3),2);
```

### Python `QRex.py`

```
from numpy.random import rand
from numpy.linalg import
matrix_rank
from scipy.linalg import qr,norm
from numpy import eye
A = rand(4,3)
A[:,2] = 2 * A[:,0]-0.3 * A[:,1]
r = matrix_rank(A)
Y, R, p =qr(A,mode='economic',
pivoting=True)
err = norm(A[:,p] - Y @ R,2)
orthchk = norm(Y.T @ Y - eye(3),2)
```

# QR factorization (continued)

Y =

```
-0.5455  0.6271  0.5234
-0.5115 -0.5767  0.3490
-0.2467  0.4553 -0.5755
-0.6164 -0.2585 -0.5225
```

P =

```
3      2      1
```

R =

```
-2.0981 -0.9605 -1.1931
         0 -0.3781 -5.6714e-02
         0      0  1.8618e-16
```

err = 6.9155e-16

orthchk = 2.3524e-16

# Singular value decomposition (SVD), $A \in \mathbb{R}^{m \times n}$ , $m \geq n$

## Definition (Singular value decomposition (SVD))

$A$  has an **SVD**, that is,

$$A = U \begin{bmatrix} \Sigma \\ \mathbf{0} \end{bmatrix} V^T,$$

where  $U \in \mathbb{R}^{m \times m}$  and  $V \in \mathbb{R}^{n \times n}$  are orthogonal, and  $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_n) \in \mathbb{R}^{n \times n}$ , is diagonal with **singular values**  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ .

- $\text{rank}(A) = \text{rank}(\Sigma) = r$ ;  $r$  is the number of positive singular values. In this case, it is sometimes useful to write

$$\begin{bmatrix} \Sigma \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix},$$

where  $\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r)$  is an invertible diagonal matrix.

# SVD, $A \in \mathbb{R}^{m \times n}$ , $m \geq n$ (continued)

Write  $U = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m]$  and  $V = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n]$ . If  $\text{rank}(A) = r$ , then

$$A = U \begin{bmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} V^T = [\mathbf{u}_1, \dots, \mathbf{u}_r] \Sigma_r \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix} = [\sigma_1 \mathbf{u}_1, \dots, \sigma_r \mathbf{u}_r] \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix}.$$

Therefore,

$$A = \sum_{j=1}^r \sigma_j \mathbf{u}_j \mathbf{v}_j^T.$$

Terminology:

- **singular values:**  $\sigma_1, \sigma_2, \dots, \sigma_n$  with  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$
- **left singular vectors:**  $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m$  ( $\mathbf{u}_i^T A = \sigma_i \mathbf{v}_i^T$ )
- **right singular vectors:**  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$  ( $A \mathbf{v}_i = \sigma_i \mathbf{u}_i$ )

# SVD (continued)

## Example (SVD MATLAB-Python discussion, numerical rank!)

### MATLAB *SVex.m*

```
% A: vander matrix with
% entries Aij=x(i)^(n-j)
% where n=101
%

x=linspace(-1,1,101);
A=vander(x);
sv=svd(A);

r = rank(A);
semilogy([1:101],sv, '.');
```

### Python *SVex.py*

```
from numpy import
linspace, vander, finfo, arange
from numpy.linalg import matrix_rank
from scipy.linalg import svd
from matplotlib.pyplot import semilogy
x = linspace(-1, 1, 101)
A = vander(x)
sv = svd(A, compute_uv=False)
eps = finfo(float).eps
r = matrix_rank(A)
semilogy(arange(1, 102), sv, '.')
```

**Note:** If  $A \in \mathbb{R}^{m \times n}$ , then in MATLAB, `rank(A)` uses a tolerance of  $\max(m, n) \text{eps}(\sigma_1)$ , while in Python `matrix_rank(A)` uses  $\max(m, n) \epsilon \sigma_1$ .

# SVD (continued)

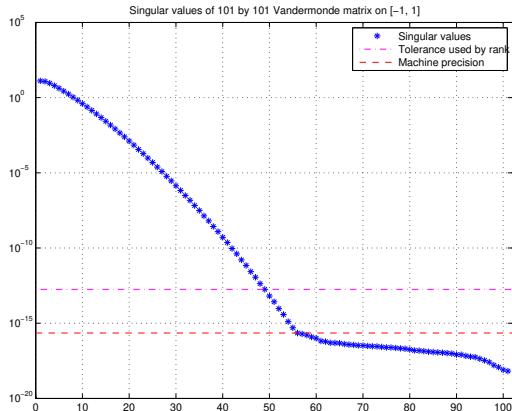
`rank(A)=48`

`sv(48)= 4.2545e-13`

`sv(49) = 1.7794e-13`

`MATLAB tol = 1.7941e-13`

`Python tol = 2.9020e-13`



# Condition number using singular values

Let  $A \in \mathbb{R}^{n \times n}$  with  $\text{rank}(A) = n$ , and let  $B = A^T A$ . Then

$$B = A^T A = V \Sigma^T U^T U \Sigma V^T = V \Sigma^2 V^T, \quad \Sigma^2 = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2).$$

This is an eigendecomposition of  $B$ . Hence, the eigenvalues of  $B$  are  $\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2$ , and consequently,

$$\|A\|_2 = \max_{1 \leq i \leq n} \sqrt{\sigma_i^2} = \sigma_1.$$

Since  $A^{-1} = V \Sigma^{-1} U^T$ , where  $\Sigma^{-1} = \text{diag}(1/\sigma_1, 1/\sigma_2, \dots, 1/\sigma_n)$ , we also have  $\|A^{-1}\|_2 = 1/\sigma_n$ . Therefore,

$$\kappa_2(A) = \sigma_1/\sigma_n \left( = \lambda_n/\lambda_1 \text{ if } A \text{ is symmetric \& positive-definite,} \right. \\ \left. \text{with eigenvalues } 0 < \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n, \text{ see Topic 02} \right).$$

**Note:** If  $A \in \mathbb{R}^{m \times n}$  for  $m > n$ , with  $\text{rank}(A) = n$ , then  $\kappa_2(A) = \sigma_1/\sigma_n$ .

• MATLAB `eig`/Python `scipy.linalg.eigh` orders  $\lambda_1 \leq \dots \leq \lambda_n$